

# SMARTBAND V001 REV2

## Full System Development Report

*Dashboards · Cloud · Hospital Portal · FHIR · Registration · Recommendations*

|               |                                             |
|---------------|---------------------------------------------|
| Student       | Masrur Jamil Prochchhod                     |
| Matriculation | 2210028                                     |
| Institution   | Hochschule Hamm-Lippstadt (HSHL)            |
| Supervisor    | Prof. Giovanni (GitHub: giovannix)          |
| Project       | Smartband V001 Rev2 — Health Wearable       |
| Report Date   | June 2026                                   |
| Platform      | Raspberry Pi 5 · Flask · Firebase Firestore |

# 1. Project Overview

This report documents the complete software development work carried out on the Smartband V001 Rev2 project during the June 2026 development session. The session produced a fully functional, multi-user health monitoring platform built on a Raspberry Pi 5, covering six distinct user-facing dashboards, a Firebase Firestore cloud backend, a FHIR R4 clinical data layer, a hospital band management portal, a patient self-registration system, and a doctor-to-patient recommendation system.

The system is accessible from any device on the local network and is designed to be sensor-agnostic — all data is currently simulated, and when the Rev2 PCB hardware arrives, replacing the simulation with real BLE sensor readings requires changing only one file.

*All six dashboards, the cloud storage, and every API endpoint described in this report are live and accessible on the Raspberry Pi at 192.168.0.110:5000. The system runs continuously as a Flask application with a background simulation thread.*

## 1.1 System at a Glance

| Component        | Technology           | Purpose                                                        |
|------------------|----------------------|----------------------------------------------------------------|
| Flask App        | Python 3.13 + Flask  | Serves all pages and REST API endpoints                        |
| Simulator        | Python threading     | Generates live sensor data for 5 patients simultaneously       |
| Cloud            | Firebase Firestore   | Persistent storage — patients, readings, alerts, registrations |
| Main Dashboard   | HTML5 + Chart.js     | Patient-facing AMOLED smartwatch companion UI                  |
| Doctor Dashboard | HTML5 + Fetch API    | Clinical multi-patient view with threshold alerts              |
| Hospital Portal  | HTML5 + LocalStorage | Band inventory management and patient allocation               |
| FHIR Layer       | FHIR R4 + LOINC      | Clinical interoperability standard — 8 LOINC observations      |
| Registration     | HTML5 + localStorage | Patient self-registration with unique Band ID generation       |

## 2. Main Dashboard — Patient Interface

The main dashboard at the root URL (/) is the patient-facing interface of the system. It is styled as an AMOLED smartwatch companion application — pure black background, Apple Watch-style per-metric colour coding, and smooth live updates every two seconds. The design philosophy mirrors real consumer health wearable companion apps such as Fitbit, Garmin Connect, and Samsung Health.

**Main Dashboard — AMOLED Patient Interface**  
URL: <http://192.168.0.110:5000/>  
*Live ECG strip at top · Core vitals (HR, SpO2, Temp, HRV) · Rev2 sensors (BP, Stress, RR, Floor) · Activity & sleep · Trend charts · Alert log · History table · Patient profile · Doctor recommendations section*

### 2.1 ECG Waveform Strip

The top of the dashboard features a live ECG strip rendered on an HTML5 canvas element. The 64-sample waveform is generated in Python using a mathematical PQRST synthesiser (Gaussian functions per cardiac component) at a simulated 256Hz rate. The canvas renderer uses the shadow API to produce a green glow effect matching real cardiac monitor displays. The waveform morphology changes automatically when the active simulation scenario is switched — faster, compressed complexes during the Workout scenario, slower during Sleep.

### 2.2 Sensor Cards

Sensor data is organised into three horizontal card rows, each refreshing independently every two seconds via JavaScript fetch calls to /api/live. The Core Vitals row shows HR (red), SpO2 (teal), Temperature (orange), and HRV (purple). The Rev2 Sensors row shows Blood Pressure with an automatic category label (Normal/Elevated/High Stage 1/High Stage 2), Stress with a sliding gradient marker on a colour spectrum bar, Respiratory Rate, and Floor/Altitude. The Activity and Sleep row shows Steps with a 10,000-step goal bar, Calories, Sleep Stage badge (colour-coded per stage), and Battery.

### 2.3 Scenario Switcher

Six scenario buttons allow instant switching of all simulated sensor values: Normal, Workout, Fever, Fall, Stairs, and Sleep. Each scenario sets all 16 sensor channels to internally consistent physiological values — for example, the Fall scenario simultaneously sets HR to 140, acceleration to 8.5G, blood pressure to 160/100, and stress to 90%, matching the acute stress response of a real fall event. Switching is done via a POST request without page reload.

### 2.4 Patient Profile Form

A collapsible patient profile section allows the patient to enter their personal details: name, gender, date of birth (auto-calculates age), weight, height, and city. These details are used throughout the

system — in the FHIR Patient resource, on the doctor dashboard, and in hospital band allocation records. The form saves immediately to the server's `active_profile` dictionary.

## 2.5 Doctor Recommendations

At the bottom of the main dashboard, above the navigation links, is a Recommendations section that displays messages sent by the doctor. Unread recommendations are highlighted with a purple notification dot and a "New" label. Recommendations show the doctor's name, timestamp, and urgency level (Routine green / Important orange / Urgent red blinking). Clicking a recommendation marks it as read. The unread count shows in a badge next to the section title. This section refreshes every 10 seconds.

## 3. Doctor Dashboard — Clinical View

The doctor dashboard at `/doctor` is a dedicated clinical interface showing all monitored patients simultaneously with live vitals, threshold alert notifications, and tools for clinical interaction. It was designed following clinical dashboard conventions — compact patient cards, colour-coded status, and immediate visibility of any patient in distress.

### Doctor Dashboard — Multi-Patient Clinical View

URL: <http://192.168.0.110:5000/doctor>

*Sticky alert banner · 4 overview stats · Patient grid with cards · Alert log panel · Detail modal on click · Add patient modal (Band ID search + manual) · Recommendation button per patient*

### 3.1 Multi-Patient Architecture

The dashboard simultaneously monitors five patients, each running their own simulated physiological scenario in the background. The simulator maintains a separate telemetry dictionary per patient, updated every two seconds. Patients 1 through 5 represent: a healthy adult (normal baseline), a patient with hypertension (BP spikes to 168/104), a post-surgery patient with fever (temp spikes to 39.4°C), a routine wellness patient, and a COPD patient (SpO2 drops to 86%). This provides realistic test coverage of all alert conditions.

### 3.2 Threshold Alert System

Seven clinical thresholds are monitored continuously: HR over 150 or below 40, SpO2 below 90%, temperature above 39.0°C, systolic BP above 160mmHg, diastolic BP above 100mmHg, stress above 80%, and respiratory rate above 25 breaths per minute. When any threshold is crossed, three things happen simultaneously: the patient card's border turns red and a blinking dot appears, the active alerts appear as red chips on the card, and if the total alert count increases, a sticky red banner appears at the top of the page with an audible beep (Web Audio API) and a browser push notification.

### 3.3 Add Patient — Band ID Search

The Add Patient modal has two tabs. The Band ID Search tab allows the doctor to type a patient's SB-XXXXXX Band ID or their name. As they type, a live search queries the registered patients database and shows matching results instantly. Selecting a result shows a preview card with all the patient's registration details. The doctor selects a simulation scenario and clicks Add — the patient appears in the grid immediately and begins being simulated. The Manual Add tab allows the doctor to enter all patient details by hand without a prior registration.

### 3.4 Doctor Recommendations

Each patient card has a purple button in the top-right corner. Clicking it opens a recommendation modal where the doctor selects an urgency level (Routine, Important, or Urgent), enters their name, and types the recommendation text. Submitting sends a POST to `/api/doctor/patient/<pid>/recommend`, which saves to `recommendations.json` and will be visible to the patient on their main dashboard within 10 seconds. A green toast notification confirms the send.

## 4. Patient Registration — Band ID System

The patient registration system provides a self-service onboarding flow for new users. A patient who receives a Smartband can register their details and receive a unique Band ID that links their personal profile to the device. The doctor can then search for this ID to add the patient to their monitoring dashboard without needing to re-enter any information.

#### Patient Registration Page

URL: `http://192.168.0.110:5000/register`

*Clean registration form · Auto-generates Band ID (SB-XXXXXX) · Copy button · Go to dashboard button · localStorage persistence · Steps for sharing ID with doctor*

### 4.1 Registration Flow

The patient navigates to `/register` from the main dashboard navigation (Register my band button) or directly via URL. They fill in their name, gender, date of birth, weight, height, city, and any medical notes. Submitting sends a POST to `/api/register` which generates a cryptographically random 6-character alphanumeric Band ID (format: SB-XXXXXX), stores the complete profile in `registered_patients.json` and Firebase Firestore, and returns the ID to the browser.

### 4.2 Band ID Persistence

After receiving their Band ID, the patient sees a success screen showing the ID prominently with a Copy button. The ID is saved to browser `localStorage` so that when the patient returns to the main dashboard, their My Band ID card shows the code automatically without re-registration. The main dashboard also has a Register / Update link so patients can update their details at any time.

## 5. Hospital Portal — Band Management

The hospital portal is a separate management interface for healthcare organisations that own multiple Smartbands. It allows a hospital to register once, manage their band inventory, allocate bands to patients, monitor those patients in real time, and release bands back to available status when monitoring ends. This workflow mirrors how real hospital equipment management systems operate.

### Hospital Registration Page

URL: <http://192.168.0.110:5000/hospital/register>

*Hospital name, city, contact person, email · Band count selector (1–100) · PIN setup · Generates HOSP-XXXXXX Hospital ID · Login tab for returning hospitals*

### Hospital Dashboard — Band Inventory

URL: <http://192.168.0.110:5000/hospital/dashboard>

*4 overview stats (Total/Available/Allocated/In Use) · Band grid colour-coded by status · Allocate modal with patient name + Band ID · Mark in use · Release button · Patient monitoring tab with live vitals*

### 5.1 Hospital Registration

A hospital registers at `/hospital/register` by providing their name, city, country, contact person, email, a PIN (minimum 4 characters), and the number of Smartbands they own. The system generates a unique Hospital ID (format: HOSP-XXXXXX) and automatically creates the specified number of band records in `hospital_bands.json` with IDs in the format `HB-{HOSPITAL_CODE}-{NUMBER}` (e.g. HB-AB1234-001). These bands start with Available status.

### 5.2 Band Lifecycle

Each band follows a three-state lifecycle. In the Available state the band is unassigned and shown in green. The doctor clicks Allocate to patient, enters the patient's name, optional SB- Band ID, and monitoring notes. The band moves to Allocated status (orange). When the patient begins wearing the device, the doctor clicks Mark as in use and the band moves to In Use status (blue). When monitoring ends, clicking Release band returns it to Available — ready for the next patient. The release date is recorded for audit purposes.

### 5.3 Patient Monitoring Tab

The hospital dashboard has a Patient Monitoring tab that shows live vitals for all currently allocated patients. It pulls data from the same `/api/doctor/patients` endpoint as the doctor dashboard, matching patients by name or ID against the allocation records. If a patient has not been added to the main monitoring system, the card shows a link to do so. This gives hospital administrators a quick overview without needing to open the full doctor dashboard.

## 6. FHIR R4 Clinical Data Layer

FHIR (Fast Healthcare Interoperability Resources) R4 is the international standard for exchanging health data between medical systems. The Smartband system implements a complete FHIR R4 data layer with eight clinically relevant observations, proper LOINC codes, UCUM units, ISO 8601 timestamps, and both individual and multi-patient bundle endpoints. Steps and calories were deliberately excluded from the FHIR bundle as they are not clinical observations.

### FHIR R4 Lab Inspector

URL: <http://192.168.0.110:5000/lab/inspect>

*Patient selector pills (All + individual) · Clinical observations grid per patient · Patient resource tab · Raw bundle JSON tab · Auto-refresh every 3 seconds · 8 LOINC-coded observations*

### 6.1 Multi-Patient FHIR

The FHIR implementation supports three access patterns. The `/api/fhir/all` endpoint returns a single FHIR R4 searchset Bundle containing all patients' observations and Patient resources — this is how hospital information systems would query the device fleet. The `/api/fhir/observation/<pid>` endpoint returns the Observation Bundle for one specific patient. The `/api/fhir/patient/<pid>` endpoint returns the FHIR Patient resource. The FHIR Lab Inspector page provides a visual three-tab interface: clinical observation cards, patient resource parser, and raw JSON bundle viewer.

### 6.2 LOINC Codes Implemented

| Clinical Observation         | LOINC Code | UCUM Unit |
|------------------------------|------------|-----------|
| Heart rate                   | 8867-4     | /min      |
| Oxygen saturation (SpO2)     | 2708-6     | %         |
| Body temperature             | 8310-5     | Cel       |
| Respiratory rate             | 9279-1     | /min      |
| Systolic blood pressure      | 55284-4    | mm[Hg]    |
| Diastolic blood pressure     | 8462-4     | mm[Hg]    |
| Stress / EDA / GSR           | 74767-6    | %         |
| Heart rate variability (HRV) | 80404-7    | ms        |

## 7. Health Summary Page

The health summary page provides historical trend analysis of recorded sensor data. It groups readings from `sensor_data.csv` by date and renders statistics, goal progress rings, trend charts, and a session log table. The page is entirely client-side — all grouping, averaging, and charting happens in the browser from the raw `/api/history` response.

### Health Summary — Trends and Goals

URL: `http://192.168.0.110:5000/summary`

*Date tabs per recorded session · Stats cards (avg/min/max per metric) · SVG ring goal gauges · 4 trend charts (HR, SpO2, Temp, Acceleration) · Session log table · All-time aggregate view*

The date tab system automatically builds a pill for each day that data has been recorded, plus an All time tab. Selecting any tab filters all statistics, goals, and charts to that day's readings. The three SVG ring gauges show Resting HR goal (target  $\leq 75$  BPM), SpO2 goal (target  $\geq 98\%$ ), and daily reading count (target 50). Trend charts use Chart.js and only appear when data from at least two different dates is available.

## 8. Firebase Firestore Cloud Storage

Firebase Firestore was selected as the cloud storage backend for three reasons: the free Spark tier provides 20,000 writes and 50,000 reads per day (more than sufficient for a five-patient prototype), the Python `firebase-admin` SDK integrates cleanly with the existing Flask and Raspberry Pi stack, and Firestore's document-collection model maps naturally to the patient-centric data structure of the project.

### 8.1 Firestore Data Structure

| Collection             | Document ID             | What is stored                                                             |
|------------------------|-------------------------|----------------------------------------------------------------------------|
| <b>patients</b>        | {pid} e.g. p001         | Full patient profile — name, age, gender, city, condition, scenario        |
| <b>sensor_readings</b> | {pid}/readings/{auto}   | Telemetry snapshot every 30s — all 16 channels except ECG waveform array   |
| <b>alerts</b>          | {auto-id}               | Threshold violations — metric, value, limit, direction, patient, timestamp |
| <b>registered</b>      | {band_id} e.g. SB-4K2M9 | Self-registered patient profiles with Band IDs                             |



## 8.2 Write Rate Management

The free Firestore quota of 20,000 writes per day is managed by rate-limiting sensor readings to one write per patient per 30 seconds. With five patients, this produces a maximum of 2 writes per minute  $\times$  60 minutes  $\times$  24 hours  $\times$  5 patients = 14,400 writes per day — safely within the free limit. Alerts are written only when the alert state changes (not on every tick), and patient profiles are written only on creation, update, or deletion.

## 8.3 Graceful Offline Fallback

The cloud module is designed to fail silently. If the credentials file is missing, firebase-admin is not installed, or the network is unavailable, the application prints a warning and continues with local JSON file storage only. No crash, no error shown to users. This means the system works identically with or without cloud connectivity, which is important during development when internet access may not always be available.

*Firestore project: smartband-hshl-5b91d · Region: europe-west3 (Frankfurt) · Credentials stored at ~/smartband/data/firebase\_credentials.json · Collections: patients, sensor\_readings, alerts, registered*

## 8.4 Cloud API Endpoints

| Endpoint                                    | Function                                                                           |
|---------------------------------------------|------------------------------------------------------------------------------------|
| <code>/api/cloud/status</code>              | Returns {"connected": true/false} — checks Firebase connectivity                   |
| <code>/api/cloud/history/&lt;pid&gt;</code> | Fetches sensor reading history for a patient from Firestore (last 100)             |
| <code>/api/cloud/alerts</code>              | Fetches alert history from Firestore (last 50)                                     |
| <code>/api/cloud/recover</code>             | Fetches all patient profiles from Firestore — disaster recovery if local file lost |

## 9. Complete API and Route Reference

### 9.1 Page Routes

| URL                       | Page                                                                     |
|---------------------------|--------------------------------------------------------------------------|
| <code>/</code>            | Main patient dashboard — live sensor data, ECG, profile, recommendations |
| <code>/doctor</code>      | Doctor clinical dashboard — multi-patient, alerts, recommendations       |
| <code>/summary</code>     | Health summary — trends, goals, session history                          |
| <code>/lab/inspect</code> | FHIR R4 lab inspector — all patients, clinical observations, raw JSON    |

|                            |                                                 |
|----------------------------|-------------------------------------------------|
| <b>/register</b>           | Patient self-registration — generates Band ID   |
| <b>/hospital/register</b>  | Hospital registration and login portal          |
| <b>/hospital/dashboard</b> | Hospital band management and patient monitoring |

## 9.2 Key API Endpoints

| Endpoint                                              | Function                                                      |
|-------------------------------------------------------|---------------------------------------------------------------|
| <b>GET /api/live</b>                                  | All 16 sensor channels for primary patient — updates every 2s |
| <b>GET /api/doctor/patients</b>                       | All patients with vitals and active alerts                    |
| <b>GET /api/doctor/alerts</b>                         | All active threshold violations across all patients           |
| <b>GET /api/doctor/patient/&lt;pid&gt;</b>            | Single patient with FHIR bundle                               |
| <b>POST /api/doctor/patients/add</b>                  | Add patient (from registration or manual)                     |
| <b>DELETE /api/doctor/patients/&lt;pid&gt;</b>        | Remove patient from monitoring                                |
| <b>POST /api/doctor/patient/&lt;pid&gt;/recommend</b> | Send doctor recommendation to patient                         |
| <b>GET /api/recommendations/&lt;pid&gt;</b>           | Get recommendations for a patient                             |
| <b>POST /api/register</b>                             | Patient self-registration — returns Band ID                   |
| <b>GET /api/lookup/&lt;band_id&gt;</b>                | Look up registered patient by Band ID                         |
| <b>GET /api/search?q=</b>                             | Search registered patients by name or ID                      |
| <b>GET /api/fhir/all</b>                              | FHIR searchset Bundle — all patients                          |
| <b>GET /api/fhir/patient/&lt;pid&gt;</b>              | FHIR Patient resource                                         |
| <b>GET /api/fhir/observation/&lt;pid&gt;</b>          | FHIR Observation Bundle for one patient                       |
| <b>POST /api/hospital/register</b>                    | Register a new hospital                                       |
| <b>POST /api/hospital/login</b>                       | Hospital login — validates PIN                                |
| <b>GET /api/hospital/&lt;id&gt;/bands</b>             | Get band inventory and stats for a hospital                   |
| <b>POST /api/hospital/bands/allocate</b>              | Allocate a band to a patient                                  |

|                                                              |                                  |
|--------------------------------------------------------------|----------------------------------|
| <b>POST</b><br><b>/api/hospital/bands/&lt;id&gt;/release</b> | Release a band back to available |
| <b>GET</b> <b>/api/cloud/status</b>                          | Firebase connection status       |

## 10. Complete File Structure

| File                                     | Purpose                                                                   |
|------------------------------------------|---------------------------------------------------------------------------|
| <b>app.py</b>                            | Flask application — all routes, API endpoints, error handler              |
| <b>core/simulator.py</b>                 | Background simulation — 5 patients, 16 channels, ECG, FHIR, Firebase sync |
| <b>core/drivers.py</b>                   | OLED display driver — luma.oled, 5 screens, progress bars                 |
| <b>core/cloud_storage.py</b>             | Firebase Firestore module — read/write patients, readings, alerts         |
| <b>core/hospital_storage.py</b>          | Hospital management — bands, allocations, recommendations                 |
| <b>config/settings.py</b>                | Global constants — thresholds, GPIO, MQTT config                          |
| <b>templates/index.html</b>              | Main AMOLED patient dashboard                                             |
| <b>templates/doctor.html</b>             | Multi-patient clinical doctor dashboard                                   |
| <b>templates/summary.html</b>            | Daily/weekly health summary with trend charts                             |
| <b>templates/fhir_view.html</b>          | FHIR R4 lab inspector — multi-patient                                     |
| <b>templates/register.html</b>           | Patient self-registration with Band ID                                    |
| <b>templates/hospital_register.html</b>  | Hospital registration and login                                           |
| <b>templates/hospital_dashboard.html</b> | Hospital band management portal                                           |
| <b>data/patients.json</b>                | Active monitored patients — persists across restarts                      |
| <b>data/registered_patients.json</b>     | Self-registered patients with Band IDs                                    |
| <b>data/hospitals.json</b>               | Registered hospital profiles                                              |
| <b>data/hospital_bands.json</b>          | Band inventory — status, allocation history                               |

|                                       |                                                    |
|---------------------------------------|----------------------------------------------------|
| <b>data/recommendations.json</b>      | Doctor recommendations to patients                 |
| <b>data/firebase_credentials.json</b> | Firebase service account key (never commit to Git) |

## 11. Next Steps

---

### 11.1 Hardware (Phase 1 — Order Now)

- nRF5340 DK development kit (~€45) — mouser.de
- J-Link EDU Mini debug probe (~€22) — shop.segger.com
- Waveshare 1.69" Touch LCD (~€20) — waveshare.com
- Install nRF Connect SDK v2.6.1 on Windows via Toolchain Manager

### 11.2 Software (Immediate)

- PDF health report export — clinical summary with patient info and FHIR observations
- SQLite database migration — replace JSON files with proper queryable database
- Flask-Login user authentication — secure all dashboards with login/password
- BLE client (bleak) — receive real sensor data from nRF5340 when hardware arrives
- Flutter mobile app — iOS/Android companion connecting directly over BLE

### 11.3 BOM Finalisation

- Finalise Rev2 PCB BOM with all 42 components
- KiCad schematic from BOM — sensor connections, power rails, RF layout
- PCB layout — 4-layer 30×35mm, analog/digital isolation, RF keepout
- Order PCB fabrication — JLCPCB with PCBA service for fine-pitch components
- Validation study — compare against Contec CMS50D+ and Omron M3 reference devices